Projet d'algèbre

May 23, 2025

1 Projet : Algèbre Linéaire Numérique Appliquée à l'Analyse de Données

Réalisé par : Abdessamad Ait Brahim
Encadré par : Dr.Abdel Louly

Ce projet individuel permet de se familiariser avec l'algèbre linéaire numérique en Python, en utilisant principalement les bibliothèques numpy et scipy. Il aborde la résolution de systèmes linéaires, les décompositions matricielles (notamment SVD et QR), ainsi que leurs applications concrètes en analyse de données.

Filière : Big Data et Intelligence Artificielle.

2 Exercice 1 : Résolution de Systèmes Linéaires

3 1.1 Méthodes Directes : Décomposition LU

[1]:
```
# importer les bibliothiques essentilles
import numpy as np
from scipy.linalg import lu, solve_triangular, cho_factor, cho_solve , qr
import time
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

4 a. Genérer une matrice A (4x4) symétrique définie positive et un vecteur b (4x1) aleatoirement sans utiliser la parametre graine de la reproductivité

[2]:
```
def generer_matrice_symetrique_definie_positive(taille):
    """
    Cette fonction accepte un paramètre :
    - taille : entier, la taille de la matrice carrée à générer.

    Elle retourne une matrice symétrique définie positive de dimensions (taille␣
 ↪x taille).
    """
```

```
    # Génère une matrice aléatoire B de taille (taille x taille)
    B = np.random.randn(taille, taille)

    # A = B  * B est symétrique et semi-définie positive
    # En ajoutant la matrice identité, on garantit que A est définie positive
    A = B.T @ B + np.eye(taille)

    return A
```

[3]:
```
def generer_vecteur_aleatoire(taille):
    """ Cette fonction génère un vecteur aléatoire de taille donnée.
    Paramètre :
    - taille : entier  - taille du vecteur.

    Retour :
    - ndarray- vecteur aéatoire . """
    return np.random.randn(taille)
```

[4]:
```
taille = 4
A = generer_matrice_symetrique_definie_positive(taille)
b = generer_vecteur_aleatoire(taille)
print("Matrice A (4x4) symétrique définie positive (aléatoire):")
print(A)

print(f"\nVecteur b (4x1) (aléatoire):")
print(b)
```

```
Matrice A (4x4) symétrique définie positive (aléatoire):
[[ 8.9386488   3.42573403  4.98115023  3.57049617]
 [ 3.42573403  3.97808088  1.30632017 -0.12717388]
 [ 4.98115023  1.30632017  5.0845665   2.91485352]
 [ 3.57049617 -0.12717388  2.91485352  4.65553525]]

Vecteur b (4x1) (aléatoire):
[-2.18121053 -0.74561331  0.91727566 -0.05961519]
```

[5]:
```
# Vérification que A est symétrique et définie positive
valeurs_propres = np.linalg.eigvals(A)
est_symetrique = np.allclose(A, A.T)
est_definie_positive = np.all(valeurs_propres > 0)
nombre_condition = np.linalg.cond(A)

print(f"\nVérification:")
print(f"- Matrice symétrique: {est_symetrique}")
print(f"- Matrice définie positive: {est_definie_positive}")
print(f"- Valeurs propres: {valeurs_propres}")
print(f"- Nombre de condition: {nombre_condition:.2e}")
```

```
Vérification:
- Matrice symétrique: True
- Matrice définie positive: True
- Valeurs propres: [15.39639365  4.65456487  1.0095611   1.59631181]
- Nombre de condition: 1.53e+01
```

5 b. Utiliser scipy.linalg.lu pour obtenir les matrices P, L, U telles que PA = LU

[6]:
```python
print("\nDÉCOMPOSITION LU AVEC SCIPY")
P, L, U = lu(A)

print("Matrice de permutation P:")
print(P)
print(f"\nMatrice triangulaire inférieure L:")
print(L)
print(f"\nMatrice triangulaire supérieure U:")
print(U)
```

```
DÉCOMPOSITION LU AVEC SCIPY
Matrice de permutation P:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

Matrice triangulaire inférieure L:
[[ 1.          0.          0.          0.        ]
 [ 0.38324965  1.          0.          0.        ]
 [ 0.55725987 -0.22614093  1.          0.        ]
 [ 0.39944473 -0.56115203  0.27017603  1.        ]]

Matrice triangulaire supérieure U:
[[ 8.9386488   3.42573403  4.98115023  3.57049617]
 [ 0.          2.66516951 -0.60270392 -1.49556529]
 [ 0.          0.          2.17247537  0.58695078]
 [ 0.          0.          0.          2.23149983]]
```

[7]:
```python
# Vérification que PA = LU
verification_PA_LU = np.allclose(P @ A, L @ U)
erreur_decomposition = np.linalg.norm(P @ A - L @ U)
print(f"\nVérification PA = LU: {verification_PA_LU}")
print(f"Erreur ||PA - LU||: {erreur_decomposition:.2e}")
```

```
Vérification PA = LU: True
Erreur ||PA - LU||: 4.44e-16
```

6 c. Résolverle système Ax = b en utilisant les matrices L et U (résolution triangulaire)

[8]:
```python
print("\nRÉSOLUTION TRIANGULAIRE")
print("Système original: Ax = b")
print("Après décomposition: PAx = Pb, soit LUx = Pb")
print("\nÉtapes de résolution:")
# Étape 1: Calculer Pb
Pb = P @ b
print(f"Étape 1 - Calcul de Pb:")
print(f"Pb = P @ b = {Pb}")

# Étape 2: Résoudre Ly = Pb (substitution avant)
y = solve_triangular(L, Pb, lower=True)
print(f"\nÉtape 2 - Résolution de Ly = Pb (substitution avant):")
print(f"y = {y}")

# Étape 3: Résoudre Ux = y (substitution arrière)
x = solve_triangular(U, y, lower=False)
print(f"\nÉtape 3 - Résolution de Ux = y (substitution arrière):")
print(f"x = {x}")

print(f"\nSOLUTION FINALE avec décomposition LU:")
print(f"x = {x}")
```

```
RÉSOLUTION TRIANGULAIRE
Système original: Ax = b
Après décomposition: PAx = Pb, soit LUx = Pb

Étapes de résolution:
Étape 1 - Calcul de Pb:
Pb = P @ b = [-2.18121053 -0.74561331  0.91727566 -0.05961519]

Étape 2 - Résolution de Ly = Pb (substitution avant):
y = [-2.18121053  0.09033486  2.15320516  0.28060503]

Étape 3 - Résolution de Ux = y (substitution arrière):
x = [-0.95062251  0.32091008  0.95715594  0.12574728]

SOLUTION FINALE avec décomposition LU:
x = [-0.95062251  0.32091008  0.95715594  0.12574728]
```

7 d. Vérifier la solution en comparant avec numpy.linalg.solve

[9]:
```python
print("\nVÉRIFICATION AVEC NUMPY.LINALG.SOLVE")
# Solution avec numpy.linalg.solve
x_direct = np.linalg.solve(A, b)
print(f"Solution avec numpy.linalg.solve:")
print(f"x_direct = {x_direct}")

# Calcul de l'erreur entre la solution LU et numpy
erreur = np.linalg.norm(x - x_direct)
erreur_relative = erreur / np.linalg.norm(x_direct)

print(f"\nComparaison des solutions:")
print(f"Erreur absolue ||x_LU - x_direct|| = {erreur:.2e}")
print(f"Erreur relative = {erreur_relative:.2e}")

# Tolérances
if erreur < 1e-12:
    print("Les solutions sont identiques (à la précision machine)")
elif erreur < 1e-10:
    print("Les solutions sont très proches")
else:
    print("Différence détectable entre les solutions")

# Vérification finale avec les résidus
print("\nVérification finale - Calcul des résidus:")
residu_LU = np.linalg.norm(A @ x - b)
residu_direct = np.linalg.norm(A @ x_direct - b)

print(f"Résidu méthode LU: ||Ax - b|| = {residu_LU:.2e}")
print(f"Résidu numpy: ||Ax - b|| = {residu_direct:.2e}")
```

```
VÉRIFICATION AVEC NUMPY.LINALG.SOLVE
Solution avec numpy.linalg.solve:
x_direct = [-0.95062251  0.32091008  0.95715594  0.12574728]

Comparaison des solutions:
Erreur absolue ||x_LU - x_direct|| = 0.00e+00
Erreur relative = 0.00e+00
Les solutions sont identiques (à la précision machine)

Vérification finale - Calcul des résidus:
Résidu méthode LU: ||Ax - b|| = 2.72e-16
Résidu numpy: ||Ax - b|| = 2.72e-16
```

8 1.2 Méthode de Cholesky

9 a. Utiliser scipy.linalg.cho_factor et scipy.linalg.cho_solve pour résoudre Ax = b.

[10]:
```
# Calcul de la factorisation de Cholesky
c, lower = cho_factor(A)  # c est la matrice factorisée, lower indique si la␣
 ↪factorisation est inférieure

# Résolution du système
x_chol = cho_solve((c, lower), b)

print("Solution par décomposition de Cholesky :")
print(x_chol)
```

```
Solution par décomposition de Cholesky :
[-0.95062251  0.32091008  0.95715594  0.12574728]
```

10 b. Comparer la précision et le temps d'exécution avec la méthode LU

[11]:
```
# Méthode LU
debut_lu = time.time()
P, L, U = lu(A)
Pb = P @ b
y = solve_triangular(L, Pb, lower=True)
x_lu = solve_triangular(U, y, lower=False)
fin_lu = time.time()
temps_lu = fin_lu - debut_lu

# Méthode Cholesky
debut_chol = time.time()
c, lower = cho_factor(A)
x_chol = cho_solve((c, lower), b)
fin_chol = time.time()
temps_chol = fin_chol - debut_chol

# Précision : différence entre les solutions
erreur = np.linalg.norm(x_lu - x_chol)

# Résidus
residu_lu = np.linalg.norm(A @ x_lu - b)
residu_chol = np.linalg.norm(A @ x_chol - b)

# Affichage résultats
print(f"Temps d'exécution LU : {temps_lu:.6f} s")
```

```
print(f"Temps d'exécution Cholesky : {temps_chol:.6f} s")
print(f"Erreur entre solutions LU et Cholesky : {erreur:.2e}")
print(f"Résidu méthode LU : {residu_lu:.2e}")
print(f"Résidu méthode Cholesky : {residu_chol:.2e}")
```

```
Temps d'exécution LU : 0.000000 s
Temps d'exécution Cholesky : 0.000000 s
Erreur entre solutions LU et Cholesky : 2.78e-16
Résidu méthode LU : 2.72e-16
Résidu méthode Cholesky : 1.07e-15
```

11 1.3 Méthodes Itératives : Jacobi et Gauss-Seidel

12 a. Fonction pour la méthode de Jacobi : décomposition et itération

[12]:
```
def methode_jacobi(A, b, tol=1e-6, max_iter=100):
    """
    Cette fonction résout le système Ax = b avec la méthode itérative de Jacobi.
    Paramètres : A : matrice carrée (n x n) dans notre cas (taille x taille)
    b : vecteur (n)
    tol : tolérance d'arrêt (critère de convergence)
    max_iter : nombre maximum d'itérations autorisées

    Retour :
    x : solution approchée
    n_iter : nombre d'itérations effectuées
    """
    n = A.shape[0]
    x = np.zeros_like(b, dtype=float)  # Initialisation de la solution (x0 = 0)

    D = np.diag(np.diag(A))            # Matrice diagonale D
    L_plus_U = A - D                   # Partie hors diagonale (L + U)

    D_inv = np.diag(1 / np.diag(D))    # Inverse de D (diagonale inversée)

    for k in range(max_iter):
        x_new = D_inv @ (b - L_plus_U @ x)  # Calcul de la nouvelle itération

        # Critère d'arrêt : si la différence max entre itérations est faible
        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            return x_new, k+1  # Convergence atteinte

        x = x_new

    return x, max_iter  # Retour si pas de convergence au bout de max_iter
```

13 b. Fonction méthode de Gauss-Seidel

[13]:
```python
def methode_gauss_seidel(A, b, tol=1e-6, max_iter=100):
    """
    cette fésout le système Ax = b avec la méthode itérative de Gauss-Seidel.

    Paramètres :
    -------------
    A : matrice carrée (n x n)
    b : vecteur (n)
    tol : tolérance d'arrêt
    max_iter : nombre max d'itérations

    Retour :
    --------
    x : solution approchée
    n_iter : nombre d'itérations effectuées
    """
    n = A.shape[0]
    x = np.zeros_like(b, dtype=float)  # Initialisation

    for k in range(max_iter):
        x_new = np.copy(x) #nouvelle version de x ....
        for i in range(n):
            somme = 0
            for j in range(n):
                if j != i:
                    # Utilise la dernière valeur connue pour la mise à jour
                    somme += A[i, j] * (x_new[j] if j < i else x[j])
            x_new[i] = (b[i] - somme) / A[i, i]

        # Critère d'arrêt : différence max entre itérations
        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
            return x_new, k+1  # Convergence

        x = x_new

    return x, max_iter  # Pas de convergence après max_iter
```

14 c. Tester les deux méthodes sur le système Ax = b avec une tolérance de 10−6 et un maximum de 100 itérations.

[14]:
```python
tol = 1e-6
max_iter = 100

# Résolution par Jacobi
```

```
x_jacobi, iter_jacobi = methode_jacobi(A, b, tol, max_iter)

# Résolution par Gauss-Seidel
x_gs, iter_gs = methode_gauss_seidel(A, b, tol, max_iter)

print("Résultats méthode Jacobi :")
print(f"Solution x = {x_jacobi}")
print(f"Nombre d'itérations = {iter_jacobi}")

print("\nRésultats méthode Gauss-Seidel :")
print(f"Solution x = {x_gs}")
print(f"Nombre d'itérations = {iter_gs}")
```

```
Résultats méthode Jacobi :
Solution x = [2.83249203e+14 2.42493612e+14 3.53640831e+14 2.98902908e+14]
Nombre d'itérations = 100

Résultats méthode Gauss-Seidel :
Solution x = [-0.95062088  0.32090883  0.9571555   0.12574626]
Nombre d'itérations = 36
```

15 d. Comparer le nombre d'itérations et la précision.

[15]:
```
# Solution exacte avec numpy.linalg.solve pour comparer
x_exact = np.linalg.solve(A, b)

# Calcul des erreurs entre solutions approchées et solution exacte
erreur_jacobi = np.linalg.norm(x_jacobi - x_exact)
erreur_gs = np.linalg.norm(x_gs - x_exact)

print("\nComparaison de la précision :")
print(f"Erreur Jacobi par rapport à la solution exacte : {erreur_jacobi:.2e}")
print(f"Erreur Gauss-Seidel par rapport à la solution exacte : {erreur_gs:.2e}")

print("\nConclusion :")
if iter_gs < iter_jacobi:
    print(f"Gauss-Seidel converge plus rapidement ({iter_gs} itérations) que␣
 ↪Jacobi ({iter_jacobi} itérations).")
else:
    print(f"Jacobi converge plus rapidement ({iter_jacobi} itérations) que␣
 ↪Gauss-Seidel ({iter_gs} itérations).")

if erreur_gs < erreur_jacobi:
    print("Gauss-Seidel fournit une solution plus précise.")
else:
    print("Jacobi fournit une solution plus précise.")
```

```
Comparaison de la précision :
Erreur Jacobi par rapport à la solution exacte : 5.95e+14
Erreur Gauss-Seidel par rapport à la solution exacte : 2.33e-06

Conclusion :
Gauss-Seidel converge plus rapidement (36 itérations) que Jacobi (100
itérations).
Gauss-Seidel fournit une solution plus précise.
```

16 Exercice 2 : Décompositions SVD et QR

17 2.1 Décomposition SVD

18 a. Créez une matrice de données D (10x5)

[16]:
```
# Simulation de données : ventes hebdomadaires (en unités) dans 10 magasins␣
↪pour 5 produits
D = np.random.randint(50, 500, size=(10, 5))

print("Matrice de données D (10 magasins × 5 produits) :")
print(D)
```

```
Matrice de données D (10 magasins × 5 produits) :
[[296 166 141 421  58]
 [396 469 403 414 258]
 [497 485  58 246 321]
 [461 395 347 185 237]
 [269 338 349 400 447]
 [242 387 390 133 331]
 [190 169 130  64 371]
 [127 330 461 376 269]
 [151 231 221 349  90]
 [372 377 275 475 418]]
```

19 b. Utiliser numpy.linalg.svd pour obtenir U, S, V

[17]:
```
# Décomposition SVD
U, S, Vt = np.linalg.svd(D, full_matrices=False)

print("\nMatrice U (10x5) :")
print(U)

print("\nValeurs singulières S (vecteur) :")
print(S)
```

```
print("\nMatrice V transposée (V ) (5x5) :")
print(Vt)
```

```
Matrice U (10x5) :
[[-0.22383763  0.24272255  0.58477451  0.04343008 -0.4254173 ]
 [-0.4004842   0.11296759  0.10414367 -0.31540138  0.10862461]
 [-0.33662973 -0.62098839  0.34561737  0.04692825  0.52454932]
 [-0.33465817 -0.26162862 -0.01995168 -0.53513041 -0.5289984 ]
 [-0.36719402  0.12556201 -0.19935351  0.41710444 -0.09847007]
 [-0.30330552 -0.09877451 -0.47642721 -0.2783459   0.11644895]
 [-0.18645327 -0.31408181 -0.33582119  0.39785705 -0.31280103]
 [-0.318629    0.48417677 -0.2999076  -0.07178232  0.23797902]
 [-0.2155038   0.33866031  0.20526772 -0.05758011  0.26711269]
 [-0.39323862  0.03099037  0.12555263  0.43801601 -0.05608129]]

Valeurs singulières S (vecteur) :
[2181.16119387  465.10685892  422.41683246  303.99359132  136.18139457]

Matrice V transposée (V ) (5x5) :
[[-0.44624077 -0.50276929 -0.41351408 -0.45192107 -0.41577117]
 [-0.51237087 -0.23741275  0.48158305  0.60145369 -0.29570679]
 [ 0.43509981 -0.01674287 -0.5204772   0.53157978 -0.50688682]
 [-0.22977522 -0.33121759 -0.46243743  0.38326378  0.6904772 ]
 [-0.54427638  0.76215013 -0.33517007  0.06747307 -0.07745204]]
```

20 c. Reconstruiser la matrice D et vérifiez l'exactitude

[18]:
```
# Conversion de S en matrice diagonale
Sigma = np.diag(S)

# Reconstruction de D
D_reconstruite = U @ Sigma @ Vt

print("\nMatrice D reconstruite à partir de U, S, V  :")
print(np.round(D_reconstruite))

# Vérification d'exactitude
erreur_reconstruction = np.linalg.norm(D - D_reconstruite)
print(f"\nErreur de reconstruction ||D - UΣV || = {erreur_reconstruction:.2e}")
```

```
Matrice D reconstruite à partir de U, S, V  :
[[296. 166. 141. 421.  58.]
 [396. 469. 403. 414. 258.]
 [497. 485.  58. 246. 321.]
 [461. 395. 347. 185. 237.]
 [269. 338. 349. 400. 447.]
```

```
 [242. 387. 390. 133. 331.]
 [190. 169. 130.  64. 371.]
 [127. 330. 461. 376. 269.]
 [151. 231. 221. 349.  90.]
 [372. 377. 275. 475. 418.]]

Erreur de reconstruction ||D - UΣV|| = 3.80e-12
```

21 d. Réduction de dimension : approximation avec les 2 premières valeurs singulières

[19]:
```python
# Réduction de dimension : k = 2
k = 2
U_k = U[:, :k]       # (10x2)
S_k = np.diag(S[:k]) # (2x2)
Vt_k = Vt[:k, :]     # (2x5)

# Approximation de D avec k=2 composantes principales
D_approx = U_k @ S_k @ Vt_k

print("\nMatrice D approximée avec les 2 premières valeurs singulières :")
print(np.round(D_approx))

# Vérification de l'erreur d'approximation
erreur_approx = np.linalg.norm(D - D_approx)
print(f"\nErreur d'approximation ||D - D_approx|| = {erreur_approx:.2e}")
```

```
Matrice D approximée avec les 2 premières valeurs singulières :
[[160. 219. 256. 289. 170.]
 [363. 427. 387. 426. 348.]
 [476. 438. 165. 158. 391.]
 [388. 396. 243. 257. 339.]
 [327. 389. 359. 397. 316.]
 [319. 344. 251. 271. 289.]
 [256. 239.  98.  96. 212.]
 [195. 296. 396. 450. 222.]
 [129. 199. 270. 307. 149.]
 [375. 428. 362. 396. 352.]]

Erreur d'approximation ||D - D_approx|| = 5.38e+02
```

22 2.2 Décomposition QR

23 a. Utiliser scipy.linalg.qr pour décomposer la matrice D en Q et R

[20]:
```
# Décomposition QR
Q, R = qr(D, mode='economic')

print("\nMatrice Q (orthogonale) :")
print(Q)

print("\nMatrice R (triangulaire supérieure) :")
print(R)
```

```
Matrice Q (orthogonale) :
[[-0.28910041  0.45260912  0.36828407  0.42358267  0.17422773]
 [-0.38676947 -0.18006964  0.05536382  0.02278834  0.33116816]
 [-0.48541522  0.10367253 -0.8039181   0.01170903  0.05435472]
 [-0.45025436  0.27073272  0.35823984 -0.53238142  0.24342681]
 [-0.26272977 -0.18407925  0.17570316  0.18391645 -0.45431553]
 [-0.23635912 -0.42943579  0.0231149  -0.42302503  0.00226243]
 [-0.18557121  0.09185063  0.0823165  -0.22579366 -0.63379082]
 [-0.1240397  -0.62912075  0.21906733  0.15438539  0.06840149]
 [-0.14748028 -0.23462624 -0.0061693   0.36595806  0.25916968]
 [-0.3633289   0.03311373  0.02170563  0.33907665 -0.34324204]]

Matrice R (triangulaire supérieure) :
[[-1023.86571385 -1066.27361892  -778.7114943   -903.63803327
   -842.11629351]
 [    0.          -314.31126227  -461.34674216  -235.98455725
   -289.62249557]
 [    0.             0.           338.55367689   215.62893891
     46.66606506]
 [    0.             0.             0.           441.83566432
    -17.34876189]
 [    0.             0.             0.             0.
   -368.52985916]]
```

24 b. Vérifier que Q est orthogonale (Q Q = I) et que QR = D

[21]:
```
# Vérification 1 : Q est orthogonale  Q @ Q = I
QTQ = Q.T @ Q
identite = np.eye(Q.shape[1])
erreur_orthogonalite = np.linalg.norm(QTQ - identite)
```

```python
print("\nVérification Q  @ Q = I :")
print(QTQ)
print(f"Erreur ||Q Q - I|| = {erreur_orthogonalite:.2e}")

# Vérification 2 : QR = D
D_reconstruit = Q @ R
erreur_reconstruction = np.linalg.norm(D - D_reconstruit)

print("\nVérification QR = D :")
print(np.round(D_reconstruit))
print(f"Erreur de reconstruction ||QR - D|| = {erreur_reconstruction:.2e}")
```

```
Vérification Q  @ Q = I :
[[ 1.00000000e+00  2.77555756e-17 -5.55111512e-17  5.55111512e-17
   8.32667268e-17]
 [ 2.77555756e-17  1.00000000e+00 -1.08420217e-18 -3.98986399e-17
  -1.09287579e-16]
 [-5.55111512e-17 -1.08420217e-18  1.00000000e+00  4.16333634e-17
   7.80625564e-18]
 [ 5.55111512e-17 -3.98986399e-17  4.16333634e-17  1.00000000e+00
   1.38777878e-17]
 [ 8.32667268e-17 -1.09287579e-16  7.80625564e-18  1.38777878e-17
   1.00000000e+00]]
Erreur ||Q Q - I|| = 6.91e-16

Vérification QR = D :
[[296. 166. 141. 421.  58.]
 [396. 469. 403. 414. 258.]
 [497. 485.  58. 246. 321.]
 [461. 395. 347. 185. 237.]
 [269. 338. 349. 400. 447.]
 [242. 387. 390. 133. 331.]
 [190. 169. 130.  64. 371.]
 [127. 330. 461. 376. 269.]
 [151. 231. 221. 349.  90.]
 [372. 377. 275. 475. 418.]]
Erreur de reconstruction ||QR - D|| = 4.80e-13
```

25 Exercice 3 : Application à l'Analyse de Données

26 3.1 Analyse en Composantes Principales (ACP)

27 a. Centrer les données (soustraire la moyenne de chaque colonne)

[22]:
```
# Centrage des données : soustraction de la moyenne colonne par colonne
D_moy = D.mean(axis=0)
D_centree = D - D_moy

print("Moyenne de chaque colonne :")
print(D_moy)

print("\nDonnées centrées :")
print(D_centree)
```

```
Moyenne de chaque colonne :
[300.1 334.7 277.5 306.3 280. ]

Données centrées :
[[  -4.1 -168.7 -136.5  114.7 -222. ]
 [  95.9  134.3  125.5  107.7  -22. ]
 [ 196.9  150.3 -219.5  -60.3   41. ]
 [ 160.9   60.3   69.5 -121.3  -43. ]
 [ -31.1    3.3   71.5   93.7  167. ]
 [ -58.1   52.3  112.5 -173.3   51. ]
 [-110.1 -165.7 -147.5 -242.3   91. ]
 [-173.1   -4.7  183.5   69.7  -11. ]
 [-149.1 -103.7  -56.5   42.7 -190. ]
 [  71.9   42.3   -2.5  168.7  138. ]]
```

28 b. appliquez la SVD sur la matrice centrée

[23]:
```
# Décomposition SVD de la matrice centrée
U, S, Vt = np.linalg.svd(D_centree, full_matrices=False)

print("\nValeurs singulières :")
print(S)
print("\nPremières composantes principales (lignes de Vt) :")
print(Vt)
```

```
Valeurs singulières :
[502.8303593  460.80218765 420.79418309 300.66459925 132.05161057]

Premières composantes principales (lignes de Vt) :
```

```
[[-0.51618838 -0.62622401 -0.32300903 -0.13334891 -0.46826946]
 [ 0.39094293  0.03817478 -0.63917096 -0.64559422  0.1427407 ]
 [ 0.46862482  0.06249655 -0.42719712  0.60479925 -0.47770854]
 [-0.19224152 -0.2573193  -0.42265292  0.44612837  0.72053033]
 [ 0.56933922 -0.73230263  0.35495463 -0.02476942  0.11392718]]
```

29 c. Projeter les données sur les deux premières composantes principales

[24]:
```
# Projection sur les 2 premières composantes principales
# U[:, :2] * S[:2] donne les coordonnées projetées
X_proj = U[:, :2] * S[:2]

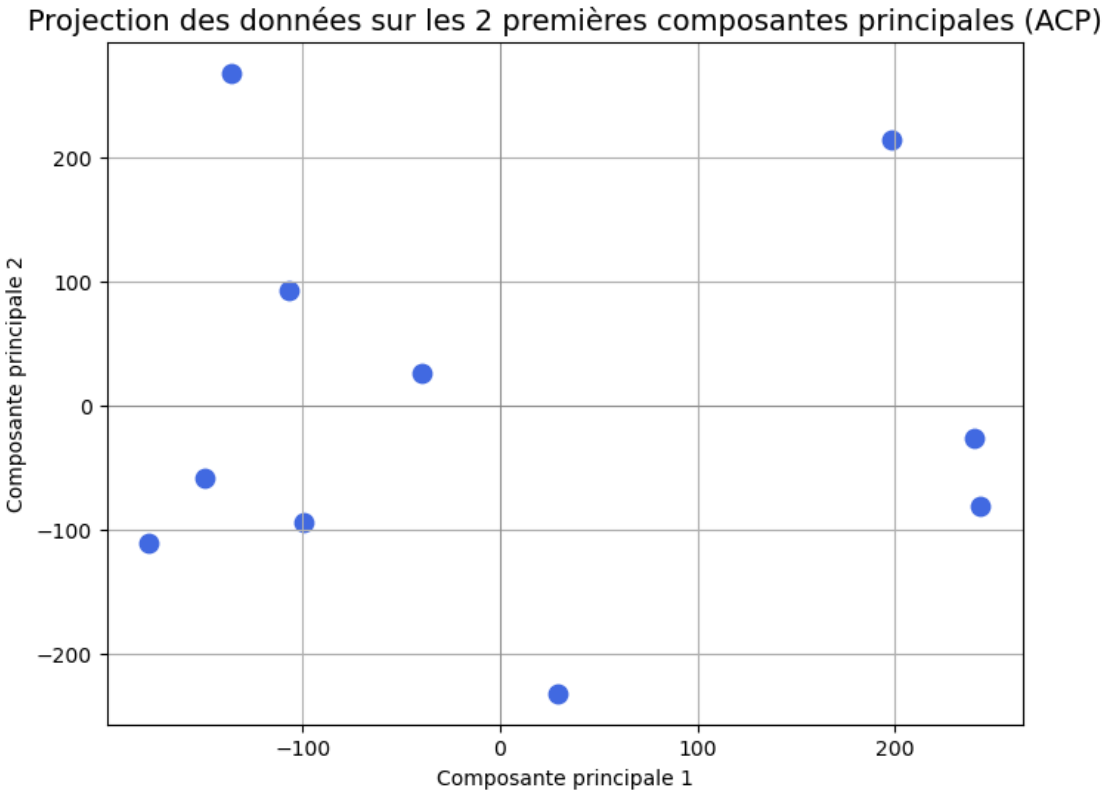
print("\nDonnées projetées sur les 2 premières composantes principales :")
print(X_proj)
```

```
Données projetées sur les 2 premières composantes principales :
[[ 240.51179431  -26.53420795]
 [-178.20173252 -110.2684467 ]
 [-136.0165871   267.79405797]
 [-106.95433664   92.9550044 ]
 [ -99.80401799  -94.38755352]
 [ -39.87186289   26.53727764]
 [ 197.93941058  214.3262195 ]
 [  28.87984989 -231.70757796]
 [ 243.43032564  -80.8227627 ]
 [-149.91284327  -57.89201068]]
```

30 d. Visualiser les données projetées avec matplotlib

[25]:
```
plt.figure(figsize=(8, 6))
plt.scatter(X_proj[:, 0], X_proj[:, 1], color='royalblue', s=80)

plt.title("Projection des données sur les 2 premières composantes principales␣
 ↪(ACP)", fontsize=14)
plt.xlabel("Composante principale 1")
plt.ylabel("Composante principale 2")
plt.grid(True)
plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.show()
```



31 3.2 Compression d'Image

32 a. Charger une image et la convertir en niveaux de gris :

[26]:
```python
def charger_et_convertir_en_gris(chemin_fichier):
    """Charge une image locale et la convertit en niveaux de gris"""
    # Charger l'image
    img = mpimg.imread(chemin_fichier)
    print(f"Image chargée - Dimensions : {img.shape}")

    # Convertir en niveaux de gris si nécessaire
    if len(img.shape) == 3:  # Image couleur (RGB)
        gris_img = 0.299*img[:,:,0] + 0.587*img[:,:,1] + 0.114*img[:,:,2]
        print("Image convertie en niveaux de gris")
    else:  # Déjà en niveaux de gris
        gris_img = img
        print("Image déjà en niveaux de gris")

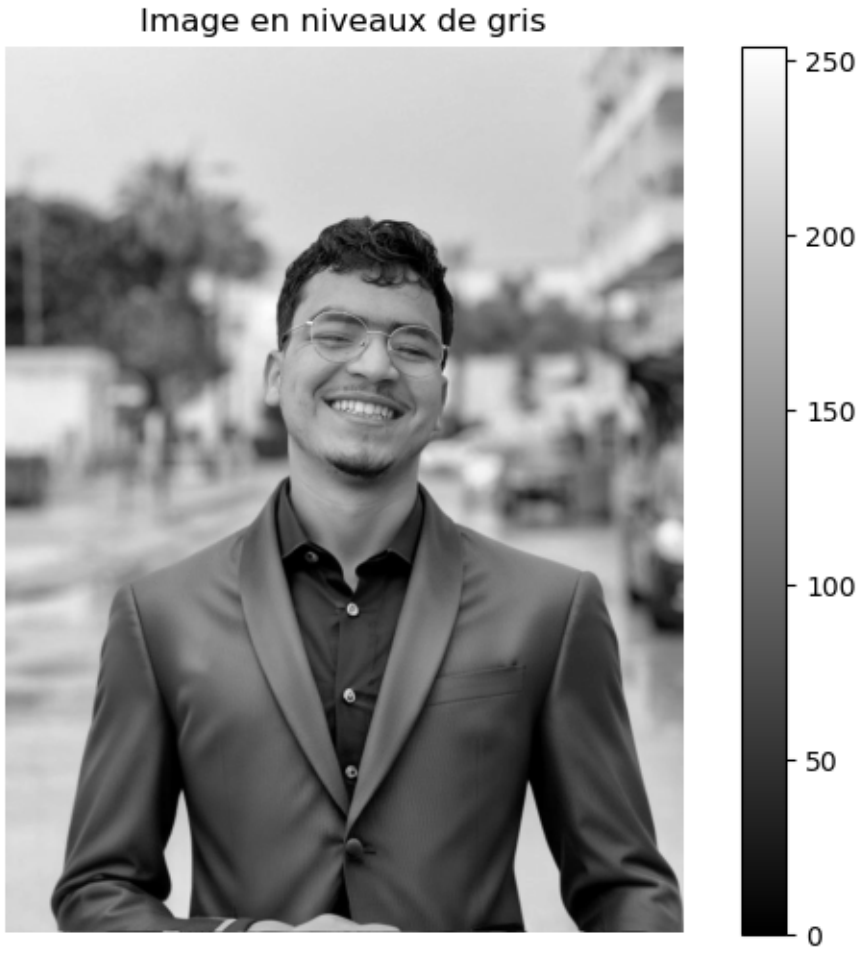
    return gris_img
```

```python
# Utilisation
chemin_image = r"C:\Users\moi\Downloads\samad.jpg"
image_gris = charger_et_convertir_en_gris(chemin_image)

# Affichage
plt.figure(figsize=(8, 6))
plt.imshow(image_gris, cmap='gray')
plt.title('Image en niveaux de gris')
plt.axis('off')
plt.colorbar()
plt.show()

print(f"Dimensions finales : {image_gris.shape}")
print(f"Valeurs min/max : {image_gris.min():.3f} / {image_gris.max():.3f}")
```

```
Image chargée - Dimensions : (1024, 781, 3)
Image convertie en niveaux de gris
```



```
Dimensions finales : (1024, 781)
Valeurs min/max : 0.000 / 254.017
```

33 b. Appliquer la SVD sur la matrice de l'image

[27]:
```python
def appliquer_svd(image):
    """Applique la décomposition en valeurs singulières (SVD) sur l'image"""
    print("Application de la SVD...")

    # SVD : A = U × Σ × V^T
    U, valeurs_singulieres, Vt = np.linalg.svd(image, full_matrices=False)

    print(f"Dimensions de U : {U.shape}")
    print(f"Nombre de valeurs singulières : {len(valeurs_singulieres)}")
    print(f"Dimensions de Vt : {Vt.shape}")

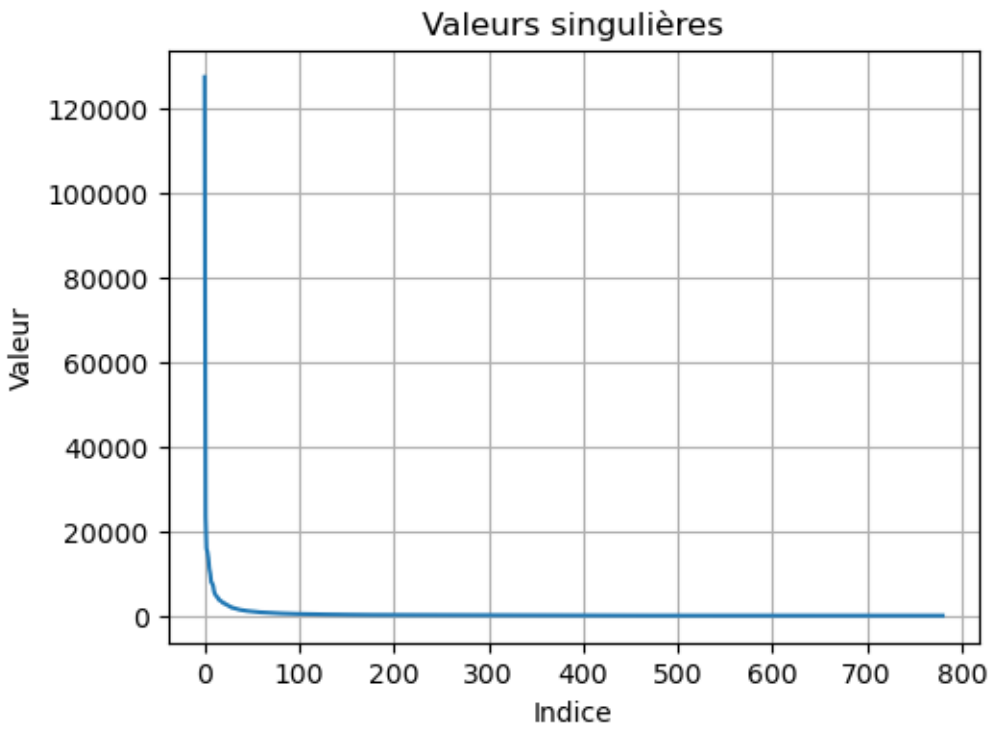
    return U, valeurs_singulieres, Vt
```

[28]:
```python
# Application de la SVD
U, s, Vt = appliquer_svd(image_gris)
```

```
Application de la SVD…
Dimensions de U : (1024, 781)
Nombre de valeurs singulières : 781
Dimensions de Vt : (781, 781)
```

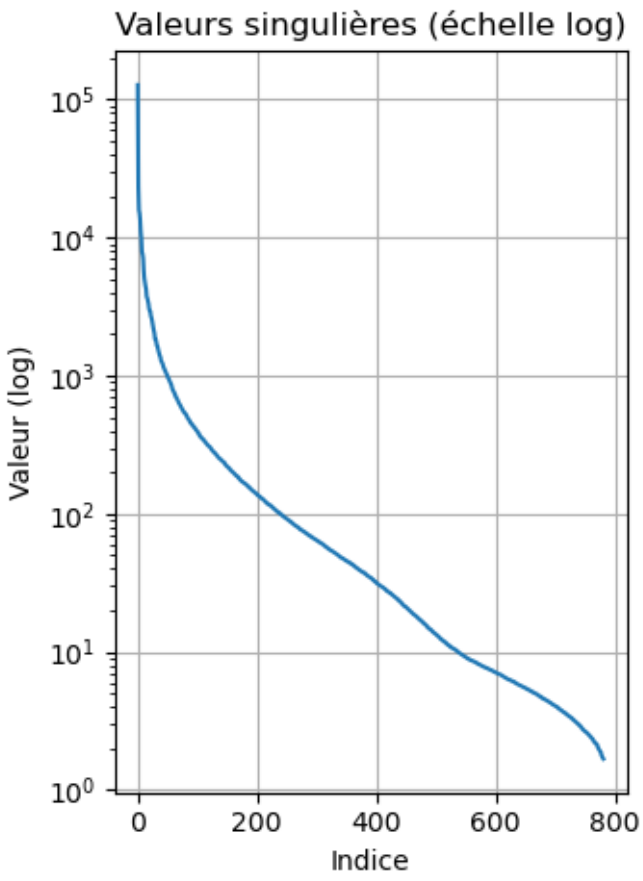
[29]:
```python
# Visualisation des valeurs singulières
plt.figure(figsize=(12, 4))

# Graphique 1 : Valeurs singulières
plt.subplot(1, 2, 1)
plt.plot(s)
plt.title('Valeurs singulières')
plt.xlabel('Indice')
plt.ylabel('Valeur')
plt.grid(True)
```



[30]:
```
# Graphique 2 : Valeurs singulières en échelle logarithmique
plt.subplot(1, 2, 2)
plt.semilogy(s)
plt.title('Valeurs singulières (échelle log)')
plt.xlabel('Indice')
plt.ylabel('Valeur (log)')
plt.grid(True)

plt.tight_layout()
plt.show()
```



[31]:
```
# Informations sur la SVD
print(f"\nInformations SVD:")
print(f"Rang de la matrice: {np.sum(s > 1e-10)}")
print(f"Plus grande valeur singulière: {s[0]:.2f}")
print(f"Plus petite valeur singulière: {s[-1]:.2e}")
print(f"Rapport max/min: {s[0]/s[-1]:.2e}")
```

```
Informations SVD:
Rang de la matrice: 781
Plus grande valeur singulière: 127457.12
Plus petite valeur singulière: 1.68e+00
Rapport max/min: 7.57e+04
```

34 c. Reconstruire l'image avec différents rangs (k = 10, 50, 100)

[32]:
```
def reconstruire_image(U, s, Vt, rang):
    """Reconstruit l'image en utilisant seulement les 'rang' premières composantes"""
    print(f"Reconstruction avec rang {rang}...")

    # Sélectionner seulement les 'rang' premières composantes
    U_tronque = U[:, :rang]
    s_tronque = s[:rang]
    Vt_tronque = Vt[:rang, :]

    # Reconstruction : A_approx = U_k × Σ_k × V_k^T
    reconstruite = U_tronque @ np.diag(s_tronque) @ Vt_tronque

    return reconstruite

def calculer_taux_compression(shape_original, rang):
    """Calcule le taux de compression en pourcentage"""
    m, n = shape_original
    taille_originale = m * n
    taille_compressee = rang * (m + n + 1)  # U_k + s_k + Vt_k
    taux = (taille_compressee / taille_originale) * 100
    return taux

# Différents rangs à tester
rangs = [10, 50, 100]
images_reconstruites = {}

# Remplacer 'image_gris' par le nom exact de ta variable d'image en niveaux de gris
for rang in rangs:
    images_reconstruites[rang] = reconstruire_image(U, s, Vt, rang)
    taux_compression = calculer_taux_compression(image_gris.shape, rang)
    print(f"Rang {rang} : {taux_compression:.1f}% de la taille originale")

print(f"\nErreurs de reconstruction (norme de Frobenius) :")
for rang in rangs:
    erreur = np.linalg.norm(image_gris - images_reconstruites[rang], 'fro')
    erreur_relative = erreur / np.linalg.norm(image_gris, 'fro') * 100
    print(f"Rang {rang} : {erreur_relative:.2f}% d'erreur relative")

print(f"\nImages reconstruites stockées dans 'images_reconstruites'")
print(f"Rangs disponibles : {list(images_reconstruites.keys())}")
```

```
Reconstruction avec rang 10…
Rang 10 : 2.3% de la taille originale
Reconstruction avec rang 50…
```

```
Rang 50 : 11.3% de la taille originale
Reconstruction avec rang 100…
Rang 100 : 22.6% de la taille originale

Erreurs de reconstruction (norme de Frobenius) :
Rang 10 : 12.94% d'erreur relative
Rang 50 : 3.91% d'erreur relative
Rang 100 : 2.00% d'erreur relative

Images reconstruites stockées dans 'images_reconstruites'
Rangs disponibles : [10, 50, 100]
```

35 d. Visualiser les résultats

[33]:
```python
def visualiser_resultats_compression(image_gris, images_reconstruites, rangs):
    """Affiche l'image originale et les reconstructions aux différents rangs"""
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    axes[0, 0].imshow(image_gris, cmap='gray')
    axes[0, 0].set_title('Image originale')
    axes[0, 0].axis('off')

    positions = [(0, 1), (1, 0), (1, 1)]
    for i, rang in enumerate(rangs):
        r, c = positions[i]
        img = images_reconstruites[rang]

        taux = (rang * (image_gris.shape[0] + image_gris.shape[1] + 1)) /␣
↪(image_gris.size) * 100
        erreur = np.linalg.norm(image_gris - img, 'fro')
        erreur_rel = erreur / np.linalg.norm(image_gris, 'fro') * 100

        axes[r, c].imshow(img, cmap='gray')
        axes[r, c].set_title(f'Rang {rang}\nCompression : {taux:.1f}%\nErreur :␣
↪{erreur_rel:.1f}%')
        axes[r, c].axis('off')

    plt.tight_layout()
    plt.show()

def tracer_analyse_compression(image_gris, images_reconstruites, rangs):
    """Trace les courbes taux de compression et erreur relative selon rang"""
    taux_compression = []
    erreurs_relatives = []

    for rang in rangs:
```

```
        taux = (rang * (image_gris.shape[0] + image_gris.shape[1] + 1)) /␣
↪(image_gris.size) * 100
        taux_compression.append(taux)

        erreur = np.linalg.norm(image_gris - images_reconstruites[rang], 'fro')
        erreurs_relatives.append(erreur / np.linalg.norm(image_gris, 'fro') *␣
↪100)

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    axes[0].plot(rangs, taux_compression, 'bo-')
    axes[0].set_xlabel('Rang')
    axes[0].set_ylabel('Taux de compression (%)')
    axes[0].set_title('Taux de compression vs Rang')
    axes[0].grid(True)

    axes[1].plot(rangs, erreurs_relatives, 'ro-')
    axes[1].set_xlabel('Rang')
    axes[1].set_ylabel('Erreur relative (%)')
    axes[1].set_title('Erreur relative vs Rang')
    axes[1].grid(True)

    plt.tight_layout()
    plt.show()

def afficher_tableau_recapitulatif(image_gris, images_reconstruites, rangs):
    """Affiche un tableau avec rang, compression, erreur et qualité"""
    print("\n" + "="*60)
    print("TABLEAU RÉCAPITULATIF DE LA COMPRESSION")
    print("="*60)
    print(f"{'Rang':<8} {'Compression (%)':<18} {'Erreur (%)':<15} {'Qualité':
↪<10}")
    print("-"*60)

    for rang in rangs:
        taux = (rang * (image_gris.shape[0] + image_gris.shape[1] + 1)) /␣
↪(image_gris.size) * 100
        erreur = np.linalg.norm(image_gris - images_reconstruites[rang], 'fro')
        erreur_rel = erreur / np.linalg.norm(image_gris, 'fro') * 100

        if erreur_rel < 5:
            qualite = "Excellente"
        elif erreur_rel < 15:
            qualite = "Bonne"
        elif erreur_rel < 30:
            qualite = "Moyenne"
        else:
```

```
            qualite = "Faible"

        print(f"{rang:<8} {taux:<18.1f} {erreur_rel:<15.1f} {qualite:<10}")

#APPLICATION
rangs = [10, 50, 100]

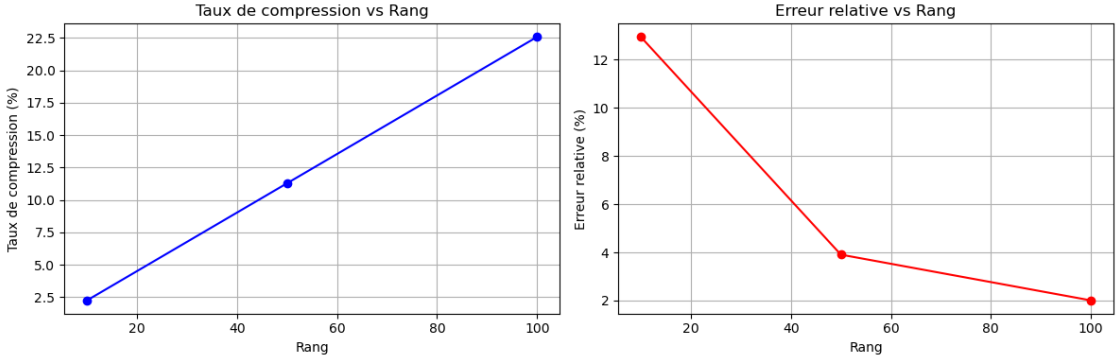
visualiser_resultats_compression(image_gris, images_reconstruites, rangs)
tracer_analyse_compression(image_gris, images_reconstruites, rangs)
afficher_tableau_recapitulatif(image_gris, images_reconstruites, rangs)
```












```
============================================================
TABLEAU RÉCAPITULATIF DE LA COMPRESSION
============================================================
Rang      Compression (%)    Erreur (%)      Qualité
------------------------------------------------------------
10        2.3                12.9            Bonne
50        11.3               3.9             Excellente
100       22.6               2.0             Excellente
```

[ ]:

[ ]: